



Universidad de Jaén

Grado en ingeniería informática

24/25 Calidad del Software

Práctica 4 - Aplicación de Métricas del Software

David Lorente Wagner
26925402S

David Fernández Espejo
77690485G

Índice

1. [Introducción](#)
2. [Estándares de Codificación](#)
 - 2.A [**DCL52-CPP**](#)
 - 2.B [**DCL53-CPP**](#)
 - 2.C [**DCL57-CPP**](#)
 - 2.D [**DCL60-CPP**](#)
 - 2.E [**ERR50-CPP**](#)
 - 2.F [**ERR60-CPP**](#)
 - 2.G [**OOP58-CPP**](#)
 - 2.H [**OOP53-CPP**](#)
 - 2.I [**OOP55-CPP**](#)
 - 2.J [**STR52-CPP**](#)
3. [Errores corregidos con CPPCheck](#)
 - 3.A [***Member variable “Starfighter:: parsecs” is not initialized in the constructor***](#)
 - 3.B [***Member variable ‘StarFighter::_parsecs’ is not assigned a value in ‘StarFighter::operator=’***](#)
 - 3.C [***Instance of ‘Piloto’ object is destroyed immediately***](#)
 - 3.D [***Class ‘Piloto’ has a constructor with 1 argument that is not explicit***](#)

- 3.E **Variable “ nombre” is assigned in constructor body. Consider performing initialization in initialization list**
- 3.F **Function parameter “nombre” should be passed by const reference**
- 3.G **Function “getNombre()” should return member “ nombre” by const reference**
- 4. **Errores analizados por CPPCheck**
 - 4.A **Out of bounds access in expression**
 - 4.B **Null pointer dereference**
 - 4.C **'operator=' should return reference to '*this'**
- 5. **Reducción de la complejidad ciclométrica**
 - 5.A **Modificación de la función Main**
- 6. **Errores de CPPCheck tras la reducción de la complejidad ciclométrica**

1. Introducción

El proyecto que hemos elegido se corresponde a la práctica 4 de **Programación Orientada a Objetos** del curso 2023/2024, en donde se introducía el concepto de las relaciones de clases de **asociación** en C++.

En esta práctica, para realizar las relaciones de asociación se ponía de ejemplo la clase “Piloto”, la cual está asociado a la clase “Droide” y “Starfighter”, es decir, que un piloto tiene un droide y una nave (Starfighter).

Por lo tanto, el objetivo principal de la práctica era que el alumno se familiarice con el uso de las **relaciones de asociación** en C++ entre clases, empleando punteros a dichas clases para realizar estas relaciones.

2. Estándares de Codificación

A continuación, vamos a **citar, explicar y mostrar** ejemplos de 10 estándares de codificación los cuales vamos a **implementar** a **nuestro proyecto** con el fin de **mejorar la calidad del software** del mismo:

2.A DCL52-CPP Never qualify a reference type with const or volatile

En C++, una **referencia** es un **alias** para otra variable, lo que significa que, cuando creas una referencia, estás creando un “apodo” para una **variable** ya existente, y no una nueva copia de ella. Por tanto, este estándar **afirma que una referencia no puede cambiar a la que está referenciando** una vez que ha sido inicializada.

Aplicación del estándar a nuestro proyecto:

Código que **NO** cumple el estándar

```
Droide& Droide::setModelo(std::string& modelo)
{
    _modelo = modelo; // Modificamos _modelo

    modelo = "Nuevo modelo"; // Modificamos la referencia 'modelo' directamente, lo cual no es correcto
    return *this;
}
```

Código que **SI** cumple el estándar

```
Droide& Droide::setModelo (const std::string& modelo )
{
    _modelo=modelo;
    return *this;
}
```

En nuestro caso, este estándar **ya estaba implementado** en nuestro proyecto, por lo que no hemos tenido que realizar modificaciones.

2.B DCL53-CPP Do not write syntactically ambiguous declarations

Este estándar consiste en **evitar las declaraciones sintácticamente ambiguas** en C++. Las declaraciones deben ser **claras y fáciles de entender para evitar confusiones**, especialmente cuando se inicializan múltiples variables en una sola línea. Utilizar una **sintaxis explícita** y separada **mejora la legibilidad y reduce errores** en el código.

Aplicación del estándar a nuestro proyecto:

Código que **NO** cumple el estándar

```
class Informe
{
private:
    static long _numInformes; ///< Variable entera que almacena el número de informes
    int _idI = 0;             ///< Variable entera que almacena un ID único asociado al informe
    int _idPiloto = 0;        ///< Variable entera que almacena un ID único asociado al piloto
    long _fechaEstelar = 0;   ///< Variable entera que almacena la fecha del informe
    std::string _datosInforme; ///< Cadena de caracteres que almacena el informe en sí mismo
```

Código que **SI** cumple el estándar

```
class Informe
{
private:
    static long _numInformes; ///< Variable entera que almacena el número de informes
    int _idI;                ///< Variable entera que almacena un ID único asociado al informe
    int _idPiloto;           ///< Variable entera que almacena un ID único asociado al piloto
    long _fechaEstelar;      ///< Variable entera que almacena la fecha del informe
    std::string _datosInforme; ///< Cadena de caracteres que almacena el informe en sí mismo

Informe::Informe() {
    _idI = 0;                // Inicializa _idI
    _idPiloto = 0;           // Inicializa _idPiloto
    _fechaEstelar = 0;       // Inicializa _fechaEstelar
    _datosInforme = "";     // Inicializa _datosInforme
    Informe(0);
}
```

En nuestro caso, este estándar **no estaba implementado** en nuestro proyecto, por lo que hemos tenido que modificar los constructores.

Las variables se **declaraban e inicializaban** en una sola línea, lo cual puede causar **ambigüedad** en la interpretación del **código**.

En la nueva versión, las variables se inicializan en el **constructor por defecto**.

2.C DCL57-CPP Do not let exceptions escape from destructors or deallocation functions

Este estándar consiste en que **no se deben permitir excepciones en destructores o funciones de liberación de memoria**. Si una excepción **escapa** de estas funciones, podría **interrumpir la liberación de recursos**, dejando el sistema en un estado **inconsistente**. Para evitarlo, **las excepciones deben manejarse internamente** o las funciones deben declararse como *noexcept*.

Aplicación del estándar a nuestro proyecto:

Código que **SI** cumple el estándar

```
for (int i = 0; i < 5; ++i) {
    try {
        size_t pos = datosPilotos[i].find(";");
        if (pos != std::string::npos) {
            pilotos[i].setNombre(datosPilotos[i].substr(0, pos));

            size_t nextPos = datosPilotos[i].find(";", pos + 1);
            if (nextPos != std::string::npos) {
                pilotos[i].setNacionalidad(datosPilotos[i].substr(pos + 1, nextPos - pos - 1));

                try {
                    pilotos[i].setNumMisiones(stoi(datosPilotos[i].substr(nextPos + 1)));
                } catch (const std::invalid_argument& e) {
                    cerr << "Error al convertir a entero en la iteración " << i << ": " << e.what() << endl;
                    pilotos[i].setNumMisiones(0);
                }
            }
        }
    }
}

StarFighter::~StarFighter ( )
{ }
```

En nuestro caso, este estándar **ya estaba implementado** en nuestro proyecto, por lo que no hemos tenido que realizar modificaciones.

Los destructores **no** lanzan excepciones. En el código proporcionado, los destructores de las clases como *Piloto*, *StarFighter*, *Droide*, etc., no contienen bloques de código que puedan lanzar excepciones, lo que **garantiza** que **las excepciones no saldrán de ellos**.

En el código principal (main), **se gestionan adecuadamente las excepciones** con bloques try-catch para capturar excepciones como `std::invalid_argument` y `std::out_of_range`, lo que garantiza que las **excepciones no interrumpen la ejecución** ni afecten la liberación de recursos.

De esta manera, se asegura que **ninguna excepción escape** de los destructores, y las excepciones en el código se **manejan de manera controlada**, cumpliendo con las buenas prácticas del estándar.

2.D DCL60-CPP Obey the one-definition rule

Este estándar consiste en **establecer que cada función o variable no-inline en un programa debe tener exactamente una definición en todo el programa**, es decir, **no puede haber más de una definición de la misma función o variable en las distintas unidades** de traducción del programa.

Aplicación del estándar a nuestro proyecto:

Código que **SI** cumple el estándar

```
class Droide
{
private:
    static int _numDroides; ///< Variable entera que almacena el número de Droides
    int _idD;                ///< Variable entera que almacena una ID Única para cada Droide
    std::string _marca;      ///< Cadena de caracteres que almacena la marca del Droide
    std::string _modelo;     ///< Cadena de caracteres que almacena el modelo del Droide

public:
    /**
     * @brief Constructor por defecto de la clase Droide
     *
     * Inicializa un objeto de la clase Droide con valores por defecto.
     */
    Droide();
};
```

En nuestro caso, este estándar **ya estaba implementado** en nuestro proyecto, por lo que no hemos tenido que realizar modificaciones.

Todas las definiciones de clases de nuestro proyecto, fueron **correctamente declaradas** en archivos de cabecera e **incluidas según correspondía**, asegurando así el **cumplimiento de la Regla de Una Definición (ODR)**. **Evitando** así conflictos por **definiciones duplicadas** o inconsistentes, lo cual puede conllevar en un **comportamiento no deseado** a la hora de la **compilación y ejecución** del programa.

2.E ERR50-CPP Do not terminate abruptly terminate the program

Este estándar consiste en **establecer que un programa en C++ no debe terminar abruptamente por excepciones que no hayan sido manejadas**. Todas las excepciones se deben capturar y manejar de **manera correcta** para evitar **cierres inesperados**. Además, también no permite lanzar excepciones en **ámbitos peligrosos**, como funciones *noexcept* o *destructores*.

Aplicación del estándar a nuestro proyecto:

Código que **NO** cumple el estándar

```
size_t nextPos = datosPilotos[i].find(";", pos + 1);
if (nextPos != std::string::npos) {
    pilotos[i].setNacionalidad(datosPilotos[i].substr(pos + 1, nextPos - pos - 1));

    try {
        pilotos[i].setNumMisiones(stoi(datosPilotos[i].substr(nextPos + 1)));
    } catch (const std::invalid_argument& e) {
        throw;
    }
}
```

Código que **SI** cumple el estándar

```
size_t nextPos = datosPilotos[i].find(";", pos + 1);
if (nextPos != std::string::npos) {
    pilotos[i].setNacionalidad(datosPilotos[i].substr(pos + 1, nextPos - pos - 1));

    try {
        pilotos[i].setNumMisiones(stoi(datosPilotos[i].substr(nextPos + 1)));
    } catch (const std::invalid_argument& e) {
        cerr << "Error al convertir a entero en la iteraci3n " << i << ": " << e.what() << endl;
        pilotos[i].setNumMisiones(0);
    }
}
```

En nuestro caso, este estándar **ya estaba implementado**, ya que como podemos ver en las capturas de pantalla, **manejamos las excepciones** con *std::invalid_argument*, **evitando** que el programa se cierre **abruptamente**. Un ejemplo de un no cumplimiento de este estándar, sería emplear *throw* para **lanzar una excepción sin manejarla**.

El **objetivo principal** es proporcionar **estabilidad y continuidad** del programa ante posibles errores.

2.F ERR60-CPP Exception objects must be nothrow copy constructible

Este estándar consiste en que **los objetos lanzados como excepciones, deben ser *nothrow copy constructible***, esto quiere decir que el **constructor copia** de un **objeto lanzado, no debe lanzar una excepción**. Además, este estándar resalta que **las excepciones deben ser capturadas por referencia** para evitar **problemas** de corte de objetos o *slicing* y así, asegurar que los objetos derivados se **traten de manera correcta**. Este estándar está relacionado con el [ERR50-DCL](#).

Aplicación del estándar a nuestro proyecto:

Código que **NO** cumple el estándar

```
try {
    pilotos[i].setNumMisiones(stoi(datosPilotos[i].substr(nextPos + 1)));
} catch (const std::invalid_argument e) {
    cerr << "Error al convertir a entero en la iteraciÃ³n " << i << ": " << e.what() <
    pilotos[i].setNumMisiones(0);
}
```

Código que **SI** cumple el estándar

```
try {
    pilotos[i].setNumMisiones(stoi(datosPilotos[i].substr(nextPos + 1)));
} catch (const std::invalid_argument& e) {
    cerr << "Error al convertir a entero en la iteraciÃ³n " << i << ": " << e.what() << endl;
    pilotos[i].setNumMisiones(0);
}
```

En nuestro caso, este estándar **ya estaba implementado**, debido a que, al realizar una **copia de la excepción**, lo hacemos con **`std::invalid_argument& e`** en lugar de **`std::invalid_argument e`**, para así **evitar** el *slicing* si se usa una **clase derivada**. Luego, de esta manera, si el **constructor copia** de **`std::invalid_argument`** **lanza una excepción**, y no hemos indicado el correspondiente **lambda**, puede llevar a un **comportamiento indefinido**.

2.G OOP58-CPP Copy operations must not mutate the source object

Este estándar consiste en que **las operaciones de copia, es decir, constructores copia y operadores de asignación, no deben modificar el objeto original**. Esto quiere decir que la copia del objeto, debe **mantener los datos originales y no alterar ningún estado del objeto copiado**. El **no cumplimiento** de esta norma puede conllevar a **comportamientos inesperados** en el código, principalmente cuando se usan **algoritmos estándar** en C++.

Aplicación del estándar a nuestro proyecto:

Código que **NO** cumple el estándar

```
Piloto& Piloto::operator = ( const Piloto& otro )
{
    if ( this != &otro )
    {
        _nombre = otro._nombre;
        _nacionalidad = otro._nacionalidad;
        _numMisiones = otro._numMisiones;
        _fechaUltimaMision = otro._fechaUltimaMision;
        _incidenciasUltimaMision = otro._incidenciasUltimaMision;
        _nave=otro._nave;
        _droide=otro._droide;
    }

    return ( *this );
}
```

Código que **SI** cumple el estándar

```
Piloto& Piloto::operator = ( const Piloto& otro )
{
    if ( this != &otro )
    {
        _nombre = otro._nombre;
        _nacionalidad = otro._nacionalidad;
        _numMisiones = otro._numMisiones;
        _fechaUltimaMision = otro._fechaUltimaMision;
        _incidenciasUltimaMision = otro._incidenciasUltimaMision;

        if (otro._nave) {
            _nave = new StarFighter(*otro._nave);
        } else {
            _nave = nullptr;
        }

        if (otro._droide) {
            _droide = new Droide(*otro._droide);
        } else {
            _droide = nullptr;
        }
    }
    return *this;
}
```

En nuestro caso, este estándar **no estaba implementado**, porque en la línea `_nave = otro._nave` y `_droide = otro.droide`, se están copiando **directamente** de los punteros, y **no** de los **objetos** a los que los **punteros apuntan**. Esto podría llevar a que si `otro._nave` u `otro._droide` son punteros a objetos, **una asignación directa de punteros no realizaría una copia en profundidad de los objetos**.

Lo que sucede es que el **puntero del objeto destino** (es decir, el actual) **apunta al mismo sitio de memoria** que el puntero del objeto original.

Esto puede **causar problemas**, ya que ambos objetos **compartirán la misma dirección de memoria**, conllevando a que si **modifica** uno de los objetos apuntados, **el otro también será modificado** y viceversa.

2.H OOP53-CPP Write constructor member initializers in the canonical order

Este estándar consiste en que **los miembros deben ser inicializados en el orden en que se declaran** y no en el orden que se presentan en la lista de inicialización del constructor.

El **objetivo principal** del estándar es **evitar problemas de inicialización de variables** antes de que estén listas o realizar una **inicialización errónea** de ciertos miembros debido a dependencias entre ellos.

Aplicación del estándar a nuestro proyecto:

Código que **NO** cumple el estándar

```
StarFighter::StarFighter(string marca, string modelo) {  
    _parsecs = 0.0; // Inicialización incorrecta al principio  
    _marca = marca; // Inicialización en orden incorrecto  
    _modelo = modelo;  
    _numStarFighters++; // Incremento de la variable estática  
    _idSF = _numStarFighters + 999; // Inicialización  
}
```

Código que **SI** cumple el estándar

```
StarFighter::StarFighter(string marca, string modelo) {  
    _marca = marca;  
    _modelo = modelo;  
    _parsecs = 0.0; // Inicialización clara de _parsecs  
    _numStarFighters++;  
    _idSF = _numStarFighters + 999;  
}
```

En nuestro caso, este estándar **ya estaba implementado**, debido a que en el constructor, los miembros **se inician en el mismo orden en el que están declarados**, esto evita que hayan **dependencias cruzadas incorrectas**.

2.I OOP55-CPP Do not use pointer-to-member operators to access nonexistent members

Este estándar consiste en **establecer que no se debe emplear una expresión de puntero a miembro**, es decir, el uso de * o -> cuando el tipo dinámico del **primer operando no tiene el miembro al que se refiere el segundo operando**. Además, también advierte que **no se debe usar un puntero a miembro nulo**, ya que esto podría llevar a un **comportamiento no definido**.

Aplicación del estándar a nuestro proyecto:

Código que **NO** cumple el estándar

```
Informe Piloto::generaInforme() const {
    Informe informe;
    informe.setIdPiloto(getIdP());
    informe.setFechaEstelar(getFechaUltimaMision()); // Devuelve el informe en formato CSV
    informe.setDatosInforme(" Nave: " + getNave().toCSV() + "\nDroide: " + getDroide().toCSV() +
        "\nIncidencias ultima sesion: " + getIncidenciasUltimaMision());
    return informe;
}
```

Código que **SI** cumple el estándar

```
Informe Piloto::generaInforme() const {
    Informe informe;
    informe.setIdPiloto(getIdP());
    informe.setFechaEstelar(getFechaUltimaMision());

    // Verificar si los punteros no son nulos antes de acceder a sus métodos
    string naveInfo = (_nave != nullptr) ? _nave->toCSV() : "Nave no disponible";
    string droideInfo = (_droide != nullptr) ? _droide->toCSV() : "Droide no disponible";

    informe.setDatosInforme(" Nave: " + naveInfo + "\nDroide: " + droideInfo + "\nIncidencias ulti
    return informe;
}
```

En nuestro caso, este estándar **no estaba implementado**, ya que, nuestro código **no verifica** si los punteros a *StarFighter* y *Droide* son nulos **antes de eliminar sus referencias**. Por tanto, **hemos agregado verificaciones de punteros nulos** en nuestro código para que sea **más seguro y cumpla** con el estándar.

2.J STR52-CPP Use valid references, pointers, and iterators to reference elements of a basic string

Este estándar consiste **en establecer que deben usarse referencias, punteros e iteradores válidos al acceder a los elementos de un `std::basic_string`**. Las operaciones que modifiquen el contenido de un `basic_string`, como por ejemplo, **pasar el objeto a funciones que puedan modificarlo o llamar a funciones miembro que no sean constantes**, pueden no validar esos punteros y referencias. Se debe considerar y **evitar el uso de punteros o referencias que no sean validadas por estas operaciones**.

Aplicación del estándar a nuestro proyecto:

Código que **NO** cumple el estándar

```
for (int i = 0; i < 5; ++i) {
    try {
        size_t pos = datosPilotos[i].find(";");
        if (pos != std::string::npos) {
            std::string nombre = datosPilotos[i].substr(0, pos);
            pilotos[i]._nombre=nombre;
        }
    }
}
```

Código que **SI** cumple el estándar

```
for (int i = 0; i < 5; ++i) {
    try {
        size_t pos = datosPilotos[i].find(";");
        if (pos != std::string::npos) {
            pilotos[i].setNombre(datosPilotos[i].substr(0, pos));
        }
    }
}
```

En nuestro caso este estándar **ya estaba implementado**, ya que, nuestro código **crea una copia válida e independiente**, sin **invalidar** ninguna **referencia ni puntero** de la cadena original, **verificando** en cada iteración, que el **carácter “;”** se encuentre en la cadena. Además, al utilizar **el método setter**, en lugar de asignarlo directamente, aseguramos **que el nombre se asigne de manera controlada**.

3. Errores corregidos con CPPCheck

Se muestra una captura de pantalla de errores detectados por **CPPCheck** en la pestaña “**Problems**”. Estos se dividen en errores de tipo **warnings** e **info**.

Description	Resource	Path	Location	Type
Member 'numPlazas' was not initialized in this constructor	StarFighter.cpp	/manolo3	line 17	Code Analysis ...
Member 'numPlazas' was not initialized in this constructor	StarFighter.cpp	/manolo3	line 20	Code Analysis ...
Member 'numMisiones' was not initialized in this constructor	Piloto.cpp	/manolo3	line 17	Code Analysis ...
Member 'numMisiones' was not initialized in this constructor	Piloto.cpp	/manolo3	line 21	Code Analysis ...
Member 'idSF' was not initialized in this constructor	StarFighter.cpp	/manolo3	line 17	Code Analysis ...
Member 'idPiloto' was not initialized in this constructor	Informe.cpp	/manolo3	line 25	Code Analysis ...
Member 'idP' was not initialized in this constructor	Piloto.cpp	/manolo3	line 17	Code Analysis ...
Member 'fechaUltimaMision' was not initialized in this constructor	Piloto.cpp	/manolo3	line 17	Code Analysis ...
Member 'fechaUltimaMision' was not initialized in this constructor	Piloto.cpp	/manolo3	line 21	Code Analysis ...
(cppcheck warning) Member variable 'StarFighter::parsecs' is not assigned a value in 'StarFighter::operator='.	StarFighter.cpp	/manolo3	line 107	cppcheck Probl...
(cppcheck warning) Member variable 'StarFighter::numPlazas' is not initialized in the constructor.	StarFighter.cpp	/manolo3	line 20	cppcheck Probl...
(cppcheck warning) Member variable 'StarFighter::idSF' is not assigned a value in 'StarFighter::operator='.	StarFighter.cpp	/manolo3	line 107	cppcheck Probl...
(cppcheck warning) Member variable 'Piloto::numMisiones' is not initialized in the constructor.	Piloto.cpp	/manolo3	line 17	cppcheck Probl...
(cppcheck warning) Member variable 'Piloto::numMisiones' is not initialized in the constructor.	Piloto.cpp	/manolo3	line 21	cppcheck Probl...
(cppcheck warning) Member variable 'Piloto::idP' is not initialized in the constructor.	Piloto.cpp	/manolo3	line 17	cppcheck Probl...
(cppcheck warning) Member variable 'Piloto::idP' is not assigned a value in 'Piloto::operator='.	Piloto.cpp	/manolo3	line 115	cppcheck Probl...
(cppcheck warning) Member variable 'Piloto::fechaUltimaMision' is not initialized in the constructor.	Piloto.cpp	/manolo3	line 17	cppcheck Probl...
(cppcheck warning) Member variable 'Piloto::fechaUltimaMision' is not initialized in the constructor.	Piloto.cpp	/manolo3	line 21	cppcheck Probl...
(cppcheck warning) Member variable 'Informe::idPiloto' is not initialized in the constructor.	Informe.cpp	/manolo3	line 25	cppcheck Probl...
(cppcheck warning) Member variable 'Informe::idP' is not assigned a value in 'Informe::operator='.	Informe.cpp	/manolo3	line 96	cppcheck Probl...
(cppcheck warning) Member variable 'Droide::idD' is not initialized in the constructor.	Droide.cpp	/manolo3	line 16	cppcheck Probl...
(cppcheck warning) Member variable 'Droide::idD' is not assigned a value in 'Droide::operator='.	Droide.cpp	/manolo3	line 75	cppcheck Probl...
(cppcheck style) Instance of 'Piloto' object is destroyed immediately.	Piloto.cpp	/manolo3	line 18	cppcheck Probl...
(cppcheck style) Instance of 'Informe' object is destroyed immediately.	Informe.cpp	/manolo3	line 22	cppcheck Probl...
(cppcheck style) Instance of 'Droide' object is destroyed immediately.	Droide.cpp	/manolo3	line 17	cppcheck Probl...
(cppcheck style) Class 'Piloto' has a constructor with 1 argument that is not explicit.	Piloto.h	/manolo3	line 56	cppcheck Probl...
i Infos (41 items)				
(cppcheck performance) Variable 'nombre' is assigned in constructor body. Consider performing initialization in initiali	Piloto.cpp	/manolo3	line 22	cppcheck Probl...
(cppcheck performance) Variable 'nombre' is assigned in constructor body. Consider performing initialization in initiali	Piloto.cpp	/manolo3	line 28	cppcheck Probl...
(cppcheck performance) Variable 'nacionalidad' is assigned in constructor body. Consider performing initialization in ir	Piloto.cpp	/manolo3	line 29	cppcheck Probl...
(cppcheck performance) Variable 'modelo' is assigned in constructor body. Consider performing initialization in initiali	Droide.cpp	/manolo3	line 22	cppcheck Probl...
(cppcheck performance) Variable 'modelo' is assigned in constructor body. Consider performing initialization in initiali	Droide.cpp	/manolo3	line 29	cppcheck Probl...
(cppcheck performance) Variable 'modelo' is assigned in constructor body. Consider performing initialization in initiali	StarFighter.cpp	/manolo3	line 22	cppcheck Probl...
(cppcheck performance) Variable 'modelo' is assigned in constructor body. Consider performing initialization in initiali	StarFighter.cpp	/manolo3	line 31	cppcheck Probl...
(cppcheck performance) Variable 'marca' is assigned in constructor body. Consider performing initialization in inicializa	Droide.cpp	/manolo3	line 21	cppcheck Probl...
(cppcheck performance) Variable 'marca' is assigned in constructor body. Consider performing initialization in inicializa	Droide.cpp	/manolo3	line 28	cppcheck Probl...
(cppcheck performance) Variable 'marca' is assigned in constructor body. Consider performing initialization in inicializa	StarFighter.cpp	/manolo3	line 21	cppcheck Probl...
(cppcheck performance) Variable 'marca' is assigned in constructor body. Consider performing initialization in inicializa	StarFighter.cpp	/manolo3	line 30	cppcheck Probl...
(cppcheck performance) Variable 'incidenciasUltimaMision' is assigned in constructor body. Consider performing initial	Piloto.cpp	/manolo3	line 32	cppcheck Probl...
(cppcheck performance) Variable 'datosInforme' is assigned in constructor body. Consider performing initialization in i	Informe.cpp	/manolo3	line 21	cppcheck Probl...
(cppcheck performance) Variable 'datosInforme' is assigned in constructor body. Consider performing initialization in i	Informe.cpp	/manolo3	line 34	cppcheck Probl...
(cppcheck performance) Function parameter 'nombre' should be passed by const reference.	Piloto.cpp	/manolo3	line 21	cppcheck Probl...
(cppcheck performance) Function parameter 'modelo' should be passed by const reference.	Droide.cpp	/manolo3	line 20	cppcheck Probl...
(cppcheck performance) Function parameter 'modelo' should be passed by const reference.	StarFighter.cpp	/manolo3	line 20	cppcheck Probl...
(cppcheck performance) Function parameter 'marca' should be passed by const reference.	Droide.cpp	/manolo3	line 20	cppcheck Probl...
(cppcheck performance) Function parameter 'marca' should be passed by const reference.	StarFighter.cpp	/manolo3	line 20	cppcheck Probl...
(cppcheck performance) Function parameter 'incidenciasUltimaMision' should be passed by const reference.	Piloto.cpp	/manolo3	line 82	cppcheck Probl...
(cppcheck performance) Function parameter 'datosInforme' should be passed by const reference.	Informe.cpp	/manolo3	line 44	cppcheck Probl...
(cppcheck performance) Function 'getNombre()' should return member 'nombre' by const reference.	Piloto.h	/manolo3	line 138	cppcheck Probl...
(cppcheck performance) Function 'getNacionalidad()' should return member 'nacionalidad' by const reference.	Piloto.h	/manolo3	line 118	cppcheck Probl...
(cppcheck performance) Function 'getModelo()' should return member 'modelo' by const reference.	Droide.h	/manolo3	line 102	cppcheck Probl...
(cppcheck performance) Function 'getModelo()' should return member 'modelo' by const reference.	StarFighter.h	/manolo3	line 113	cppcheck Probl...
(cppcheck performance) Function 'getMarca()' should return member 'marca' by const reference.	Droide.h	/manolo3	line 123	cppcheck Probl...
(cppcheck performance) Function 'getMarca()' should return member 'marca' by const reference.	StarFighter.h	/manolo3	line 133	cppcheck Probl...
(cppcheck performance) Function 'getIncidenciasUltimaMision()' should return member 'incidenciasUltimaMision' by cc	Piloto.h	/manolo3	line 168	cppcheck Probl...

Como podemos observar, disponemos de **4 warnings** y **3 infos** de distinto tipo por corregir. A continuación, vamos a corregir los problemas detectados por **CPPCheck**, mostrando una captura de pantalla del código con el **problema**, y posteriormente, otra con el código **corregido**.

3.A Member variable “Starfighter::_parsecs” is not initialized in the constructor

La advertencia indica que “_parsecs” **no se inicializa en el constructor** de “StarFighter”. Esto puede causar que los atributos de la clase **tengan un valor indefinido**, por lo tanto, lo **solucionamos** inicializando cada uno de los atributos en la **lista de inicialización** o en la **declaración**.

Esto ocurre en **todos los atributos** de las clases de nuestro proyecto, por lo que lo hemos **corregido** en cada una de ellas.

Código **SIN** corregir

```
StarFighter::StarFighter()  
{ }
```

Código **corregido**

```
StarFighter::StarFighter()  
{  
    _numPlazas = 0;  
    _parsecs = 0.0;  
    _idSF = _numStarFighters + 999;  
}
```

3.B Member variable 'StarFighter::_parsecs' is not assigned a value in 'StarFighter::operator='

Este problema indica que la variable miembro “_parsecs” **no se asigna dentro del operador de asignación**

“StarFighter::operator=” de la clase “StarFighter”. Esto puede ocasionar que el objeto **no copie correctamente** su estado desde otro objeto, por lo tanto, lo **solucionamos** asegurándonos que todos los **atributos importantes** son **copiados** dentro del **operador**.

Esto ocurre en **diferentes operadores** de las clases de nuestro proyecto, por lo que lo hemos **corregido** en cada una de ellos.

Código **SIN** corregir

```
StarFighter& StarFighter::operator = (const StarFighter& otro)
{
    if ( this != &otro )
    {
        _marca = otro._marca;
        _modelo = otro._modelo;
        _numPlazas = otro._numPlazas;
    }

    return ( *this );
}
```

Código **corregido**

```
StarFighter& StarFighter::operator = (const StarFighter& otro)
{
    if ( this != &otro )
    {
        _marca = otro._marca;
        _modelo = otro._modelo;
        _numPlazas = otro._numPlazas;
        _parsecs = otro._parsecs;
        _idSF = otro._idSF;
    }

    return ( *this );
}
```

3.C Instance of 'Piloto' object is destroyed immediately

Este problema indica que **el constructor está creando una instancia temporal** de "Piloto", el cual se destruye **inmediatamente** sin ningún propósito, ya que **no es necesario** crear esa instancia temporal, simplemente **asigna** dichos valores **directamente** al objeto de la clase en el constructor.

Esto **ocurre** en las clases: "Piloto", "Informe" y "Droide", por lo que también lo hemos **corregido** en estas clases.

Código **SIN** corregir

```
Piloto::Piloto() {  
    _numMisiones = 0;  
    _idP = _numPilotos;  
    _fechaUltimaMision = "";  
    Piloto(""); // Llama al constructor con parámetros  
}
```

Código **corregido**

```
Piloto::Piloto() {  
    _numMisiones = 0;  
    _idP = _numPilotos;  
    _fechaUltimaMision = "";  
}
```

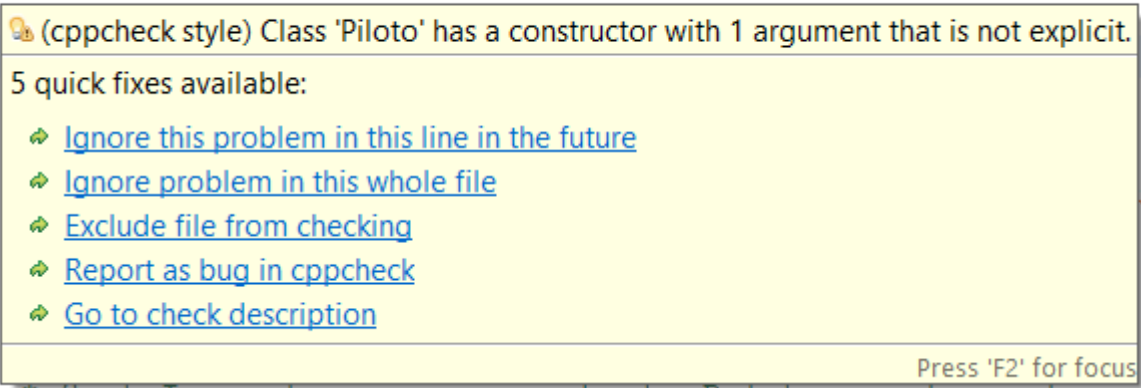
3.D Class 'Piloto' has a constructor with 1 argument that is not explicit

Este problema indica que **un constructor con un único argumento puede ser utilizado de manera implícita** para **convertir** un tipo de dato cuando se pasa como **parámetro** en una **llamada** a la **función**.

Para **solucionar** este error, incluimos la palabra clave **explicit** en el **constructor** de la clase, **evitando** que se realicen **conversiones implícitas** de tipos. Esto significa que el constructor **sólo** podrá ser llamado **explícitamente** y no será usado en **conversiones implícitas**, lo cual ayuda a **evitar errores** de este tipo.

Código **SIN** corregir

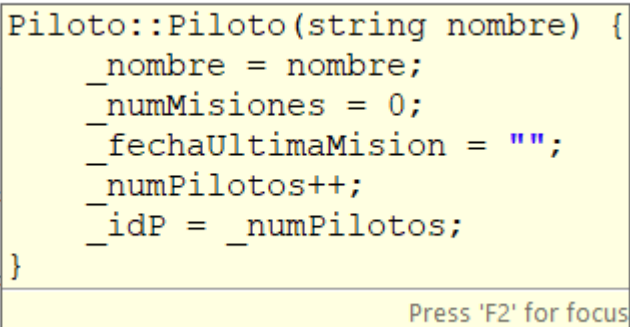
```
*/  
Piloto(std::string nombre);
```



```
*/  
//out: Inicializa un nuevo objeto Piloto con los valores
```

Código **corregido**

```
explicit Piloto(std::string nombre);  
  
/**  
 * @brief  
 *  
 * Crea u  
 *  
 * @param  
 *  
 * @in Un objeto Piloto para copiar sus valor
```



3.E Variable “_nombre” is assigned in constructor body. Consider performing initialization in initialization list

Este problema indica que **la variable “_nombre” se asigna dentro del cuerpo del constructor**. Esto puede ocasionar una pérdida de rendimiento, debido a la doble asignación. Por lo tanto, para **solucionar** este error, debemos **inicializar** las variables de la clase en la **lista de inicialización del constructor**.

Código **SIN** corregir

```
Piloto::Piloto() {
    _numMisiones = 0;
    _idP = _numPilotos;
    _fechaUltimaMision = "";
}

Piloto::Piloto(string nombre) {
    _nombre = nombre;
    _numMisiones = 0;
    _fechaUltimaMision = "";
    _numPilotos++;
    _idP = _numPilotos;
}
```

Código **corregido**

```
Piloto::Piloto()
: _numMisiones(0), _idP(++_numPilotos), _fechaUltimaMision("") {
    // No es necesario asignar nada más en el cuerpo del constructor
}

Piloto::Piloto(string nombre)
: _nombre(nombre), _numMisiones(0), _fechaUltimaMision(""), _idP(++_numPilotos) {
    // No es necesario asignar nada más en el cuerpo del constructor
}

Piloto::Piloto(const Piloto& orig)
: _nombre(orig._nombre),
  _nacionalidad(orig._nacionalidad),
  _numMisiones(orig._numMisiones),
  _fechaUltimaMision(orig._fechaUltimaMision),
  _incidenciasUltimaMision(orig._incidenciasUltimaMision),
  _nave(orig._nave ? new StarFighter(*orig._nave) : nullptr),
  _droide(orig._droide ? new Droide(*orig._droide) : nullptr),
```

3.F Function parameter “nombre” should be passed by const reference

Este error indica que el **parámetro “nombre” debería ser pasado por referencia constante**, es decir (const string&), en lugar de **por valor**, ya que no hacerlo puede causar **sobrecargas innecesarias de memoria**. Al pasarlo de esta forma, se **mejora la eficiencia** y se **evitan** copias innecesarias.

Código **SIN** corregir

```
/**
 * @brief Constructor que inicializa el piloto con un nombre
 *
 * @param nombre Nombre del piloto a asignar
 *
 * @in Nombre del piloto.
 * @out Inicializa un nuevo objeto Piloto con el nombre proporcionado.
 */
explicit Piloto(std::string nombre);
```

Código **corregido**

```
/**
 * @brief Constructor que inicializa el piloto con un nombre
 *
 * @param nombre Nombre del piloto a asignar
 *
 * @in Nombre del piloto.
 * @out Inicializa un nuevo objeto Piloto con el nombre proporcionado.
 */
explicit Piloto(const std::string& nombre);
```

3.G Function “getNombre()” should return member “_nombre” by const reference

Este error indica que la función “getNombre()” debería devolver el miembro “_nombre” por referencia constante. Esto puede ocasionar una **inconsistencia** de los datos, corriendo el **riesgo** de que el valor **pueda ser modificado desde fuera de la clase**. Por lo tanto, es conveniente **devolver** el valor por **referencia constante** para **solucionar** este error.

Código **SIN** corregir

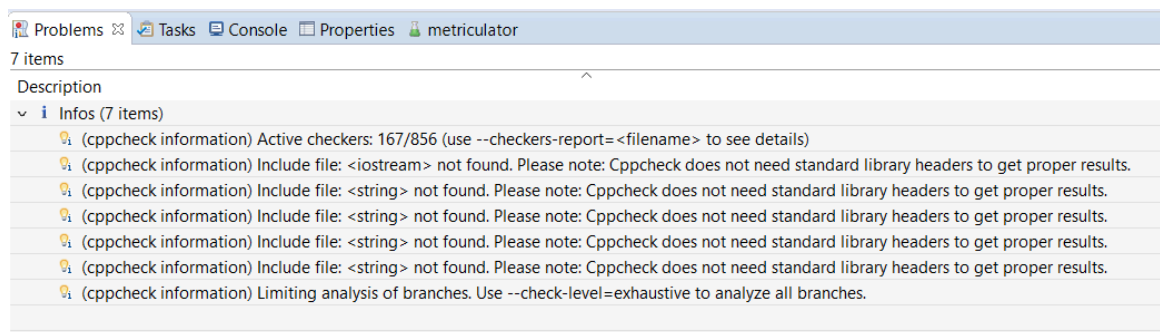
```
/**
 * @brief Obtiene el nombre del piloto
 *
 * @return Cadena con el nombre del piloto
 *
 * @in Ningún parámetro de entrada.
 * @out El nombre del piloto.
 */
std::string getNombre() const;
```

Código **corregido**

```
/**
 * @brief Obtiene el nombre del piloto
 *
 * @return Cadena con el nombre del piloto
 *
 * @in Ningún parámetro de entrada.
 * @out El nombre del piloto.
 */
const std::string& getNombre() const;
```

4. Errores analizados por CPPCheck

A continuación se muestra una **captura de pantalla** de la pestaña “**Problems**” de eclipse, donde podemos observar que **ya no tenemos errores disponibles para solucionar**, ya que, los que aparecen en la captura de pantalla, **no son relevantes**.



Como debemos detectar **10 errores**, vamos a buscar **3 errores** más en la pestaña **Project > Properties > cppcheclipse > Problems** y seleccionar **3** más para llegar a los **10**.

4.A Out of bounds access in expression

Este error analizado por **CPPCheck** se refiere a cuando **se intenta acceder a un índice de un array o vector fuera de su rango válido**, excediendo su capacidad máxima. Esto puede ocasionar un comportamiento **inesperado** o incluso **fallos** en el programa.

✓ Out of bounds access in expression Error

En nuestro proyecto, **no se produce este error**, debido a que los arrays son **controlados y no exceden de rango**. Al realizar bucles, *for (int i = 0; i < 5; ++i)*, se **asegura** que el índice **i** se mantenga entre 0 y 4 con “i=0; i < 5”.

En los arrays de “datosPilotos”, “datosNaves” y “datosDroides”, los cuales contienen **5 elementos** cada uno, se **evita** el acceso **fuera de límite** gracias a lo previamente mencionado.

Por lo tanto, nuestro proyecto **cumple con el buen uso de índices en arrays y vectores**, asegurando de que no se acceda a elementos **fuera del vector**, **evitando** así el error “**Out of bounds access in expression**”.

4.B Null pointer dereference

Este error analizado por **CPPCheck** se refiere a cuando **se intenta acceder a un puntero sin verificar si es nulo**. Esto puede conllevar a **fallos** de ejecución, como **segmentaciones o violaciones de acceso**, además de generar comportamientos **indefinidos y vulnerabilidades** de seguridad.

 Null pointer dereference Error

En nuestro proyecto, **no se produce este error**, debido a que nos aseguramos que los punteros no sean nulos antes de usarlos. Al usar:

```
if (_droide) {  
    _droide -> toCSV();  
}
```

Con este código, nos estamos **asegurando** que el **puntero** “_droide” **no es nulo** antes de acceder a sus métodos.

Por lo tanto, nuestro proyecto **cumple con el buen uso de punteros**, al **garantizar** que todos los punteros son cuidadosamente **verificados antes de ser utilizados**.

Implementamos comprobaciones (*if*) para **asegurarnos** que no se realicen acceso a **punteros nulos**, lo cual **previene** errores como la desreferenciación de punteros nulos “**null pointer dereference**”.

4.C 'Operator=' should return reference to '*this'

Este error analizado por **CPPCheck** se refiere a cuando se sobrecarga el **operador de asignación (operator=)** en una clase, pero no se **retorna una referencia al propio objeto (*this)**. Esta mala práctica puede generar comportamientos incorrectos al encadenar asignaciones, ya que el operador no devuelve el objeto original sino una copia, rompiendo así la semántica esperada en C++.

 'operator=' should return reference Warning

En nuestro proyecto, **no se produce este error**, ya que al sobrecargar el operador de asignación, se devuelve correctamente una referencia al objeto actual. Por ejemplo:

```
Droide& Droide::operator = ( const Droide& otro )
{
    if ( this != &otro )
    {
        _marca = otro._marca;
        _modelo = otro._modelo;
        _idD = otro._idD;
    }

    return ( *this );
}
```

Con este código, nos estamos **asegurando** de cumplir con las **normas del lenguaje**, permitiendo que expresiones como **a = b = c**; funcionen correctamente, ya que **operator=** retorna una **referencia válida** al objeto original.

Por lo tanto, nuestro proyecto **cumple** con las **buenas prácticas de sobrecarga de operadores**, garantizando que el **operador de**

asignación devuelva siempre ***this por referencia**. Esto evita **errores de diseño**, mantiene la **coherencia del lenguaje** y mejora la **seguridad y estabilidad** del código frente a asignaciones encadenadas.

5. Reducción de la complejidad ciclométrica

En primer lugar, vamos a realizar un **análisis** con *MetriCulator* para calcular la **complejidad ciclométrica** (McCabe) y el LSLOC, con el fin de si disponemos de módulos cuya complejidad ciclométrica sea **mayor que 10**, debemos de **modificar** dichas funciones para que su McCabe sea **menor que 10**.

Si **no disponemos** de ninguna función que **exceda** de **10** su **complejidad ciclométrica**, **modificaremos** aquellas funciones que **superen** un valor de **25 de LSLOC**, de manera que su valor sea **menor a 25**.

Scope (125 items)	NbParams	EfferentCoupli...	McCabe	NbMembers	LSLOC
● main (int argc, char** a...	2	0	12	0	53
● & Piloto::operator = (c...	1	0	4	0	15
● Piloto::Piloto(const Pilo...	1	0	3	0	1
● Piloto::generalInforme()...	0	0	3	0	8
● & Droide::operator = (...	1	0	2	0	6
● & Informe::operator = (...	1	0	2	0	7
● StarFighter::incrementa...	1	0	2	0	4

Como podemos observar, solo disponemos de **1 módulo** cuya complejidad ciclométrica **es superior a 10**, el cual es la función *Main*.

5.A Modificación de la función Main

Como la función *Main* dispone de un **McCabe de 12**, es decir, es **mayor que 10**, vamos a **modificarlo** para **reducir** su complejidad ciclomática y que sea **menor que 10**.

Scope (125 items)	NbParams	EfferentCoupli...	McCabe	NbMembers	LSLOC
● main (int argc, char** a...	2	0	12	0	53

Función Main antes de la modificación:

```
int main ( int argc, char** argv )
{
    string datosPilotos[5] = {"Dameron Poe;Jakku;100;123456;---",
                             "Anakin Skywalker;Tatooine;133;126756;Derribados 3 cazas enemigos",
                             "Han Solo;Corellia;247;126786;Impacto en motor izquierdo",
                             "Chewbacca;Kashyyk;115;136786;---",
                             "Leia Organa;Alderaan;3;126336;Derribados 10 cazas enemigos"};

    string datosNaves[5] = {"Kuat Systems;Delta-7;1",
                           "Kuat Systems;Delta-7B;1",
                           "Incom Corporation;T-65B X-wing;1",
                           "Koensayr Manufacturing;Y-wing;2",
                           "Kuat Systems;A-wing;1"};

    string datosDroides[5] = {"Industrial Automation;R2-D2",
                              "Industrial Automation;R2-KT",
                              "AccuTronics;R3-D3",
                              "AccuTronics;R3-KT",
                              "Arakyd Industries;C3PO"};

    StarFighter naveHan("Destructor", "Imperial");
    Piloto han("Han");
    han.setNave(&naveHan);
    cout << "Informacion de la nave del piloto: " << naveHan.toCSV() << endl;

    Piloto pilotos[5];
    StarFighter naves[5];
    Droide droides[5];

    for (int i = 0; i < 5; ++i) {
        try {
            size_t pos = datosPilotos[i].find(";");
            if (pos != std::string::npos) {
                pilotos[i].setNombre(datosPilotos[i].substr(0, pos));
                size_t nextPos = datosPilotos[i].find(";", pos + 1);
                if (nextPos != std::string::npos) {
                    pilotos[i].setNacionalidad(datosPilotos[i].substr(pos + 1, nextPos - pos - 1));

                    try {
                        pilotos[i].setNumMisiones(stoi(datosPilotos[i].substr(nextPos + 1)));
                    } catch (const std::invalid_argument& e) {
                        cerr << "Error al convertir a entero en la iteraciÃ³n " << i << ": " << e.what() << endl;
                        pilotos[i].setNumMisiones(0);
                    }
                }
            }

            size_t posNaves = datosNaves[i].find(";");
            if (posNaves != std::string::npos) {
                naves[i].setMarca(datosNaves[i].substr(0, posNaves));

                size_t nextPosNaves = datosNaves[i].find(";", posNaves + 1);
                if (nextPosNaves != std::string::npos) {
                    naves[i].setModelo(datosNaves[i].substr(posNaves + 1, nextPosNaves - posNaves - 1));

                    try {
                        naves[i].setNumPlazas(stoi(datosNaves[i].substr(nextPosNaves + 1)));
                    } catch (const std::invalid_argument& e) {
                        cerr << "Error al convertir a entero en la iteraciÃ³n " << i << ": " << e.what() << endl;
                        naves[i].setNumPlazas(0);
                    }
                }
            }
        }
    }
}
```

```

    }
}

size_t posDroides = datosDroides[i].find(";");
if (posDroides != std::string::npos) {
    droides[i].setMarca(datosDroides[i].substr(0, posDroides));

    size_t nextPosDroides = datosDroides[i].find(";", posDroides + 1);
    if (nextPosDroides != std::string::npos) {
        droides[i].setModelo(datosDroides[i].substr(posDroides + 1, nextPosDroides - posDroides - 1));
    }
}

} catch (const std::out_of_range& e) {
    cerr << "Índice fuera de límites en la iteración " << i << ": " << e.what() << endl;
} catch (const std::bad_alloc& e) {
    cerr << "Error de asignación de memoria en la iteración " << i << ": " << e.what() << endl;
}

pilotos[0].setDroide(&droides[2]).setNave(&naves[1]);
pilotos[1].setDroide(&droides[0]).setNave(&naves[3]);
pilotos[2].setDroide(&droides[1]).setNave(&naves[0]);

Informe informePiloto1 = pilotos[0].generaInforme();
Informe informePiloto2 = pilotos[1].generaInforme();

cout << "Informe del Piloto 1: " << informePiloto1.toCSV() << endl;
cout << "Informe del Piloto 2: " << informePiloto2.toCSV() << endl;

return 0;
}

```

Reducimos la complejidad ciclomática **modificando** el Main, **trasladando** algunas tareas a **funciones externas**. De esta forma, delegamos las asignaciones de objetos a funciones **específicas**, lo que hace que el código sea más **organizado** y **fácil de seguir**.

Este **cambio** no solo hizo que el código fuera **más fácil de leer**, sino que también ayudó a que fuera **más fácil encontrar errores**. Como cada **función** tiene una **tarea específica**, si algo sale mal, es **más sencillo** identificar el problema y **corregirlo** sin tener que **revisar todo el código** en busca de **posibles fallos**.

Función Main tras la modificación:

```

/**
 * @brief Función para asignar los datos del piloto
 *
 * Asigna el nombre, nacionalidad y número de misiones a un piloto a partir de una cadena de texto con los datos.
 *
 * @param piloto Objeto Piloto a asignar los datos.
 * @param datosCadena Cadena de texto que contiene los datos del piloto.
 */
void asignarPiloto(Piloto& piloto, const string& datosCadena) {
    size_t pos = datosCadena.find(";");
    if (pos != string::npos) {
        piloto.setNombre(datosCadena.substr(0, pos));
        size_t nextPos = datosCadena.find(";", pos + 1);
        if (nextPos != string::npos) {
            piloto.setNacionalidad(datosCadena.substr(pos + 1, nextPos - pos - 1));
            try {
                piloto.setNumMisiones(stoi(datosCadena.substr(nextPos + 1)));
            } catch (const std::invalid_argument& e) {
                cerr << "Error al convertir a entero: " << e.what() << endl;
                piloto.setNumMisiones(0);
            }
        }
    }
}

/**
 * @brief Función para asignar los datos de la nave
 *
 * Asigna la marca, modelo y número de plazas a una nave a partir de una cadena de texto con los datos.
 *
 * @param nave Objeto StarFighter a asignar los datos.
 * @param datosCadena Cadena de texto que contiene los datos de la nave.
 */
void asignarNave(StarFighter& nave, const string& datosCadena) {
    size_t pos = datosCadena.find(";");
    if (pos != string::npos) {
        nave.setMarca(datosCadena.substr(0, pos));
        size_t nextPos = datosCadena.find(";", pos + 1);
        if (nextPos != string::npos) {
            nave.setModelo(datosCadena.substr(pos + 1, nextPos - pos - 1));
            try {
                nave.setNumPlazas(stoi(datosCadena.substr(nextPos + 1)));
            } catch (const std::invalid_argument& e) {
                cerr << "Error al convertir a entero: " << e.what() << endl;
                nave.setNumPlazas(0);
            }
        }
    }
}

```

```

/**
 * @brief Función para asignar los datos del droide
 *
 * Asigna la marca y el modelo a un droide a partir de una cadena de texto con los datos.
 *
 * @param droide Objeto Droide a asignar los datos.
 * @param datosCadena Cadena de texto que contiene los datos del droide.
 */
void asignarDroide(Droide& droide, const string& datosCadena) {
    size_t pos = datosCadena.find(";");
    if (pos != string::npos) {
        droide.setMarca(datosCadena.substr(0, pos));
        size_t nextPos = datosCadena.find(";", pos + 1);
        if (nextPos != string::npos) {
            droide.setModelo(datosCadena.substr(pos + 1, nextPos - pos - 1));
        }
    }
}

/**
 * @brief Función principal del programa
 *
 * Inicializa y prueba los métodos de las clases principales. Crea instancias de pilotos, naves y droides, asigna
 * valores a sus atributos y genera informes sobre los pilotos.
 *
 * @param argc Número de argumentos de línea de comandos
 * @param argv Array de cadenas con los argumentos de línea de comandos
 * @return Código de salida del programa (0 si se ejecuta correctamente)
 */
int main(int argc, char** argv) {
    // Datos de prueba
    string datosPilotos[5] = {"Dameron Poe;Jakku;100;123456;---",
                             "Anakin Skywalker;Tatooine;133;126756;Derribados 3 cazas enemigos",
                             "Han Solo;Corellia;247;126786;Impacto en motor izquierdo",
                             "Chewbacca;Kashyyk;115;136786;---",
                             "Leia Organa;Alderaan;3;126336;Derribados 10 cazas enemigos"};

    string datosNaves[5] = {"Kuat Systems;Delta-7;1",
                           "Kuat Systems;Delta-7B;1",
                           "Incom Corporation;T-65B X-wing;1",
                           "Koenigsr Manufacturing;Y-wing;2",
                           "Kuat Systems;A-wing;1"};

    string datosDroides[5] = {"Industrial Automation;R2-D2",
                              "Industrial Automation;R2-KT",
                              "AccuTronics;R3-D3",
                              "AccuTronics;R3-KT",
                              "Arakyd Industries;C3PO"};

    // Instanciación de objetos
    StarFighter naveHan("Destructor", "Imperial");
    Piloto han("Han");
    han.setNave(&naveHan);
    cout << "Informacion de la nave del piloto: " << naveHan.toCSV() << endl;

    Piloto pilotos[5];

    Piloto pilotos[5];
    StarFighter naves[5];
    Droide droides[5];

    // Asignación de datos a los pilotos, naves y droides
    for (int i = 0; i < 5; ++i) {
        try {
            asignarPiloto(pilotos[i], datosPilotos[i]);
            asignarNave(naves[i], datosNaves[i]);
            asignarDroide(droides[i], datosDroides[i]);
        } catch (const exception& e) {
            cerr << "Error en la iteración " << i << ": " << e.what() << endl;
        }
    }

    // Asignación de droides y naves a los pilotos
    pilotos[0].setDroide(&droides[2]).setNave(&naves[1]);
    pilotos[1].setDroide(&droides[0]).setNave(&naves[3]);
    pilotos[2].setDroide(&droides[1]).setNave(&naves[0]);

    // Generación de informes
    Informe informePiloto1 = pilotos[0].generaInforme();
    Informe informePiloto2 = pilotos[1].generaInforme();

    // Mostrar informes
    cout << "Informe del Piloto 1: " << informePiloto1.toCSV() << endl;
    cout << "Informe del Piloto 2: " << informePiloto2.toCSV() << endl;

    return 0;
}

```

Realizando nuevamente un **análisis** con el *Metriculator* para **comprobar** la complejidad ciclomática tras las **modificaciones**, obtenemos los **siguientes resultados**:

● main(int argc, char** ar...	2	0	3	0	26
-------------------------------	---	---	---	---	----

Como podemos observar, la **complejidad ciclomática** (es decir, el McCabe) **se ha reducido de 12 a 3**, por lo tanto, el McCabe es **menor que 10**.

6. Errores de CPPCheck tras la reducción de la complejidad ciclomática

Una vez hemos **modificado** la función *Main* para **reducir** la **complejidad ciclomática**, han aparecido **nuevos errores detectados** por *CPPCheck*, los cuales son los siguientes:

	(cppcheck style) Variable 'datosDroides' can be declared as const	main.cpp	/manolo4	line 110	cppcheck Problem
	(cppcheck style) Variable 'datosNaves' can be declared as const	main.cpp	/manolo4	line 105	cppcheck Problem
	(cppcheck style) Variable 'datosPilotos' can be declared as const	main.cpp	/manolo4	line 100	cppcheck Problem

El **error** de *CPPCheck* indica que la **variable** “datosDroides”, “datosNaves” y “datosPilotos” puede **declararse** como *const* porque **no está siendo modificada** después de su **inicialización**. Esto **mejora** la claridad del código, **evitando** cambios accidentales y **permitiendo** optimizaciones por parte del **compilador**.

Código **SIN** corregir

```
int main ( int argc, char** argv )
{
    string datosPilotos[5] = { "Dameron Poe;Jakku;100;123456;---",
                              "Anakin Skywalker;Tatooine;133;126756;Derribados 3 cazas enemigos",
                              "Han Solo;Corellia;247;126786;Impacto en motor izquierdo",
                              "Chewbacca;Kashyyk;115;136786;---",
                              "Leia Organa;Alderaan;3;126336;Derribados 10 cazas enemigos" };

    string datosNaves[5] = { "Kuat Systems;Delta-7;1",
                            "Kuat Systems;Delta-7B;1",
                            "Incom Corporation;T-65B X-wing;1",
                            "Koensayr Manufacturing;Y-wing;2",
                            "Kuat Systems;A-wing;1" };

    string datosDroides[5] = { "Industrial Automation;R2-D2",
                              "Industrial Automation;R2-KT",
                              "AccuTronics;R3-D3",
                              "AccuTronics;R3-KT",
                              "Arakyd Industries;C3PO" };
```

Código corregido

```
int main(int argc, char** argv) {  
    // Datos de prueba  
    const string datosPilotos[5] = {"Dameron Poe;Jakku;100;123456;---",  
                                     "Anakin Skywalker;Tatooine;133;126756;Derribados 3 cazas enemigos",  
                                     "Han Solo;Corellia;247;126786;Impacto en motor izquierdo",  
                                     "Chewbacca;Kashyyk;115;136786;---",  
                                     "Leia Organa;Alderaan;3;126336;Derribados 10 cazas enemigos"};  
  
    const string datosNaves[5] = {"Kuat Systems;Delta-7;1",  
                                   "Kuat Systems;Delta-7B;1",  
                                   "Incom Corporation;T-65B X-wing;1",  
                                   "Koensayr Manufacturing;Y-wing;2",  
                                   "Kuat Systems;A-wing;1"};  
  
    const string datosDroides[5] = {"Industrial Automation;R2-D2",  
                                     "Industrial Automation;R2-KT",  
                                     "AccuTronics;R3-D3",  
                                     "AccuTronics;R3-KT",  
                                     "Arakyd Industries;C3PO"};
```